



Epicurius

Aplicação Android de Receitas

Vasco Costa
Carolina Tavares

Orientador: Artur Ferreira

Relatório beta realizado no âmbito de Projeto e Seminário,
da licenciatura em Engenharia Informática e de Computadores

Junho 2025

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

Epicurius

Aplicação Android de Receitas

49412 Vasco Fonseca Amado da Costa

49465 Carolina Martins Tavares

Orientador: Artur Jorge Ferreira

Relatório beta realizado no âmbito de Projeto e Seminário,
da licenciatura em Engenharia Informática e de Computadores

Junho 2025

Resumo

Este projeto tem como objetivo desenvolver uma plataforma que torne a experiência culinária mais acessível, dinâmica e personalizada, contribuindo para combater o estigma associado à complexidade da atividade culinária, ao mesmo tempo que promove a redução do desperdício alimentar.

A solução proposta consiste no desenvolvimento de uma aplicação *Android* dedicada à exploração, preparação e partilha de receitas, com uma interface simples, intuitiva e centrada no utilizador. A aplicação inclui um *feed* interativo e personalizado, sugestões diárias de menu, planeamento de refeições e uma funcionalidade de gestão do frigorífico, que permite ao utilizador registar os produtos disponíveis e receber alertas inteligentes sobre prazos de validade iminentes.

Além disso, com o objetivo de simplificar a descoberta de receitas, a aplicação disponibiliza a funcionalidade de captura de ingredientes, que permite a identificação automática de alimentos e a sugestão de receitas compatíveis com os mesmos.

No lado do servidor, é utilizada a linguagem de programação *Kotlin* em conjunto com a framework *Spring Boot*, proporcionando uma solução robusta, modular e escalável. O sistema recorre a duas bases de dados: *PostgreSQL* e *Firestore*. O *PostgreSQL* é responsável pelo armazenamento de dados relacionais, essenciais e de menor volume, otimizando o desempenho das consultas; já o *Firestore* é utilizado para gerir dados não relacionais e de grande volume, assegurando flexibilidade do armazenamento.

Por fim, a interface do utilizador será desenvolvida para a plataforma *Android*, recorrendo ao *Jetpack Compose*. Esta abordagem permite uma criação mais eficiente e reativa dos componentes visuais para interfaces nativas, devido à implementação intuitiva, simples e moderna.

Palavras-chave: *Android*, *Kotlin*, *PostgreSQL*, *Firestore*, receitas, gestão do frigorífico, sugestões diárias de menu, planeamento de refeições, *Feed*

Abstract

This project aims to develop a platform that makes the culinary experience more accessible, dynamic, and personalized, aiming to break the stigma associated with the complexity of cooking while also promoting the reduction of food waste.

The proposed solution consists of the development of an Android application dedicated to the exploration, preparation, and sharing of recipes, with a simple, intuitive, and user-centered interface. The application includes an interactive and personalized feed, daily menu suggestions, meal planning, and a fridge management feature, which allows users to register available products and receive smart alerts about upcoming expiration dates.

Additionally, to simplify recipe discovery, the app offers an ingredient capture functionality that enables automatic identification of food items and suggests compatible recipes accordingly.

On the server side, the system uses the Kotlin programming language together with the Spring Boot framework, providing a robust, modular, and scalable solution. The system employs two databases: PostgreSQL and Firestore. PostgreSQL is responsible for storing essential, low-volume relational data, optimizing query performance, while Firestore manages large-scale, non-relational data, ensuring storage flexibility.

Finally, the user interface will be developed for the Android platform using Jetpack Compose. This approach allows for more efficient and reactive creation of visual components for native interfaces, thanks to its intuitive, simple, and modern implementation.

Keywords: *Android, Kotlin, PostgreSQL, Firestore, recipes, fridge management, daily menu suggestions, meal planning, Feed*

Índice

1	Introdução	1
1.1	Motivação	1
1.2	Requisitos Funcionais	1
1.3	Requisitos Não Funcionais	3
1.4	Organização do Relatório	3
2	Arquitetura	5
2.1	Componentes Principais	5
2.2	Tecnologias	6
3	Backend	9
3.1	Organização do Software	9
3.2	Modelo Relacional	9
3.2.1	User	10
3.2.2	Token	11
3.2.3	Recipe	11
3.2.4	Ingredients	12
3.2.5	Collection	13
3.2.6	Fridge	13
3.2.7	MealPlanner	14
3.3	Modelo Físico	14
3.3.1	Aspetos e Restrições de Design	14
3.4	Repositório	17
3.4.1	Acessos	17
3.5	Serviços	20
3.6	HTTP	22
3.6.1	Gestão de Endpoints e Routing	22
3.6.2	Desserialização do Corpo do Pedido	23
3.7	Especificação OpenAPI	24
3.8	Autenticação	27

3.8.1	Geração do Token	27
3.8.2	Gestão do Ciclo de Vida do Token	28
3.8.3	Fluxo	28
3.8.4	Integração com Cookies	29
3.9	Tratamento de erros	31
3.10	Cloud Function	33
3.10.1	Gatilho	33
3.10.2	API Vision	33
3.10.3	Deploy	35
3.11	Testes	37
4	Conclusões	39
4.1	Requisitos Funcionais Concluídos	39
4.2	Próximos Passos	40
	Referências	44
A	Modelos de Dados	45

Lista de Figuras

2.1	Principais componentes do Sistema	6
3.1	Diagrama Entidade-Associação do User	10
3.2	Diagrama Entidade-Associação de Token	11
3.3	Diagrama Entidade-Associação de Recipe	12
3.4	Diagrama Entidade-Associação de Ingredients	13
3.5	Diagrama Entidade-Associação da Collection	13
3.6	Diagrama Entidade-Associação de Fridge	14
3.7	Diagrama Entidade-Associação de Meal Planner	14
3.8	Imagem enviada para a Cloud Function com o objetivo de identificar ingredientes.	34
A.1	Modelo Entidade-Associação	46
A.2	Modelo Físico	47

Listagens

3.1	Transaction	17
3.2	Transaction Manager	17
3.3	Jdbi Token Repository	18
3.4	Firestore Transaction Manager	18
3.5	Cloud Storage	19
3.6	Http Client Configurer	19
3.7	Exemplo de uma classe dos Serviços	20
3.8	Exemplo de uma função do serviço <code>UserService</code>	21
3.9	Exemplo de routing	22
3.10	Exemplo de routing com um parâmetro na <i>path</i>	22
3.11	Exemplo de um método com corpo no pedido	23
3.12	Criação e codificação do Token	27
3.13	Sha256 Token Encoder	28
3.14	Token Scheduler	28
3.15	Authentication Interceptor	29
3.16	Funções para adicionar e remover cookies de autenticação	30
3.17	Exemplo de um Exception Handler	31
3.18	Exemplo de resposta de erro indicando que o utilizador já existe no sistema.	32
3.19	Validação do método HTTP na Cloud Function	33
3.20	Possível resposta da Cloud Function com base na imagem enviada	34
3.21	Comando para o <i>deploy</i> da Cloud Function na GCP	35
3.22	Exemplo de teste unitário para adição de uma receita a uma coleção	37

Lista de Tabelas

3.1	Rotas da API de autenticação	24
3.2	Rotas da API relacionadas com o utilizador	24
3.3	Rotas da API relacionadas com o frigorífico do utilizador	25
3.4	Rotas da API relacionadas com receitas	25
3.5	Rotas da API relacionadas com ingredientes	25
3.6	Rotas da API relacionadas com coleções	26
3.7	Rotas da API relacionadas com o planeamento de refeições	26
3.8	Rotas da API relacionadas com o menu diário	26

Siglas e Acrónimos

API Application Programming Interface

DAW Desenvolvimento de Aplicações Web

DBMS Database Management System

GCP Google Cloud Platform

HTTP Hypertext Transfer Protocol

HTTPS Hyper Text Transfer Protocol Secure

JSON JavaScript Object Notation

REST API Representational State Transfer Application Programming Interface

SQL Structured Query Language

URI Uniform Resource Identifier

Capítulo 1

Introdução

Este capítulo é dedicado a explicar a nossa motivação para este projeto e a sua organização.

1.1 Motivação

A gastronomia é essencial ao dia a dia e a tecnologia tem revolucionado a forma como exploramos, preparamos e compartilhamos receitas. Ao mesmo tempo, o desperdício alimentar continua a ser um problema, com alimentos a serem frequentemente descartados por má gestão ou por falta de ideias sobre como os aproveitar. A motivação para o desenvolvimento desta aplicação visa tornar a experiência culinária mais acessível, dinâmica e personalizada, reduzindo o desperdício.

No mercado, já existem algumas soluções que procuram responder a este tipo de problema, nomeadamente a aplicação **Mr. Cook** [1]. Esta permite o armazenamento de receitas, criação de listas de compras e planeamento de refeições. No entanto, o foco está sobretudo no armazenamento de receitas e na criação de planos semanais, com menor ênfase na personalização da experiência e na gestão dos alimentos disponíveis em casa.

Este projeto propõe uma solução mais avançada: criar uma aplicação que reúna, num só espaço, funcionalidades de descoberta, planeamento e partilha de receitas, com um *feed* personalizado, sugestões diárias, planeamento de refeições e uma funcionalidade de frigorífico com alertas sobre produtos prestes a expirar. Para além disso, destaca-se a funcionalidade de identificação de ingredientes a partir de uma fotografia e sugestão de receitas baseadas no que o utilizador tem disponível, promovendo um maior aproveitamento alimentar.

1.2 Requisitos Funcionais

Para o desenvolvimento da aplicação, identificam-se os seguintes requisitos funcionais:

- **Perfil do Utilizador** - Cada utilizador possui um perfil onde é possível visualizar as suas receitas publicadas, os seus seguidores, os utilizadores que segue, e organizar as suas receitas de forma personalizada.

- **Configurações do Utilizador** - Permite ao utilizador gerir informações pessoais (nome, email e password), ajustar definições de privacidade e, se desejado, eliminar a sua conta.
- **Feed Interativo** - A aplicação apresenta um feed dinâmico com receitas publicadas pelos utilizadores seguidos, ordenadas cronologicamente com base na data de publicação.
- **Perfil de Receita** - Cada receita tem uma página dedicada com toda a informação relevante, incluindo o autor, tipo de cozinha, tipo de refeição, ingredientes, modo de preparação, entre outros.
- **Publicação de Receitas** - Os utilizadores podem criar e publicar novas receitas diretamente a partir do seu perfil, preenchendo os campos obrigatórios como título, ingredientes, preparação, tipo de refeição, entre outros.
- **Acompanhamento de uma Receita** - Qualquer utilizador pode acompanhar uma receita passo a passo através de uma interface interativa que apresenta as instruções de preparação de forma sequencial avançando apenas quando desejar. Caso o utilizador não disponha de todos os ingredientes necessários, a aplicação sugere alternativas. Em opção, é possível gerar automaticamente uma lista de compras com os ingredientes em falta, que pode ser exportada ou consultada mais tarde. No final do processo, o utilizador tem a possibilidade de avaliar a receita.
- **Pesquisa de Receitas** - A plataforma permite aos utilizadores procurar receitas utilizando:
 1. O nome da receita;
 2. Um formulário de pesquisa com filtros específicos (ex.: tipo de cozinha, tempo de preparação, ingredientes);
 3. Uma imagem captada ou carregada pelo utilizador, analisada pela aplicação para identificar os ingredientes presentes.
- **Favoritos** - Cada utilizador possui uma secção dedicada onde pode reunir as suas receitas preferidas de outros utilizadores, permitindo a criação de coleções personalizadas para uma organização mais eficiente.
- **Sugestão Diária de um Menu** - A aplicação fornece diariamente sugestões de um menu completo que inclui pequeno-almoço, almoço, lanche, jantar, sopa e sobremesa.
- **Planeamento de Refeições** - Funcionalidade que permite ao utilizador organizar refeições para dias específicos da semana, com a possibilidade de definir metas calóricas diárias.

- **Funcionalidade de Frigorífico** - A plataforma permite aos utilizadores adicionar produtos que têm em casa, com indicação de quantidades e datas de validade. O sistema notifica o utilizador quando um produto se aproxima da data de expiração.

1.3 Requisitos Não Funcionais

Como requisito não funcional identifica-se a facilidade de manutenção e atualização, permitindo a adição de novas funcionalidades e a correção de erros de forma eficiente.

1.4 Organização do Relatório

Este relatório está organizado em 4 capítulos. No Capítulo 2, é discutida a arquitetura do projeto, incluindo o diagrama de blocos, a organização do *backend* e as tecnologias utilizadas tanto no *frontend* como no *backend*.

O Capítulo 3 detalha os principais aspetos do desenvolvimento do *backend*, essenciais para compreender a arquitetura e o funcionamento do sistema. Começa com uma visão geral da estrutura do projeto e do esquema relacional da base de dados, seguida da interação do *backend* com a base de dados. A metodologia de repositório utilizada para o acesso aos dados e a camada de serviços responsável pela lógica de negócio são explicadas. A API HTTP aborda os *handlers* dos *endpoints* e o sistema de *routing*, enquanto a autenticação descreve a geração e validação de *tokens*. São também apresentados o tratamento de erros, a gestão de dados dos utilizadores e das receitas. A Cloud Function e os processos de criação de receitas, coleções, planeamento de refeições e gestão do frigorífico são explorados, bem como as estratégias de testes e o processo de implantação do *backend*.

O capítulo final, Capítulo 4, apresenta as conclusões do relatório. Resume os requisitos funcionais implementados e aponta direções para trabalho futuro.

Capítulo 2

Arquitetura

Neste capítulo, será apresentada a arquitetura geral do projeto, incluindo os principais componentes e as suas interações, bem como as tecnologias utilizadas e as razões que motivaram a sua escolha.

2.1 Componentes Principais

Na Figura 2.1, está ilustrada a arquitetura da aplicação, destacando as duas secções principais: a Aplicação Móvel (*frontend*) e o *backend*. O *frontend* é responsável por apresentar a interface da aplicação ao utilizador, enquanto que o *backend* gere a lógica de negócio e o armazenamento de dados.

O *frontend* do sistema é composto por uma aplicação móvel que permite ao utilizador interagir com todas as funcionalidades disponibilizadas pelo sistema, através da comunicação com o *backend*.

O *backend* é composto por uma *REST API*¹ que gere a lógica de negócio, processa as requisições dos clientes e comunica com as bases de dados e serviços externos.

¹”*Representational State Transfer Application Programming Interface é uma API que segue os princípios de design do estilo arquitetural REST que consiste num conjunto de regras e diretrizes como a utilização de métodos HTTP explícitos , comunicação sem estado (stateless), identificação dos recursos por URLs, e manipulação dos recursos através de representações (normalmente JSON ou XML).*”

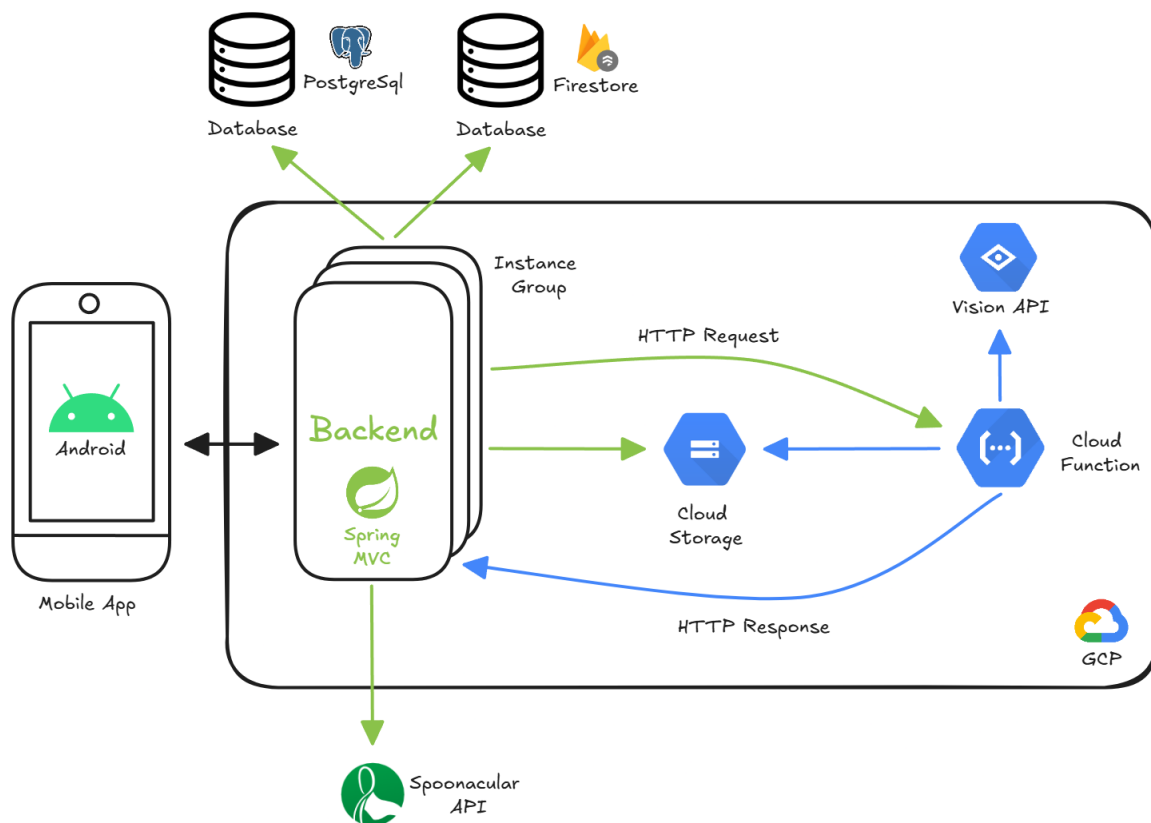


Figura 2.1: Principais componentes do Sistema

Adicionalmente, o sistema inclui um bloco de serviços em cloud providenciado pela Google Cloud Platform (GCP). O *backend* interage com o **Cloud Storage** [2] para o armazenamento eficiente de imagens. O sistema também conta com uma **Cloud Function**[3] que integra a **API Vision** [4], permitindo a análise de imagens para detecção de ingredientes. Para garantir escalabilidade e alta disponibilidade, o *backend* está implementado dentro de um grupo de instâncias, que gere o balanceamento de carga e o escalonamento automático das instâncias.

Por fim o sistema é composto por dois DBMs²: **PostgreSQL** [6] e **Firestore** [7]. O *backend* é responsável por interagir com ambas, utilizando o **PostgreSQL** para o armazenamento relacional dos dados principais e o **Firestore** para guardar informação que consome mais memória, por exemplo, os procedimentos das receitas, com o objetivo de tirar carga do **PostgreSQL** e manter a performance das *queries* relacionais.

2.2 Tecnologias

No *backend*, o framework utilizado foi o **Spring Boot** [8] em conjunto com a linguagem de programação **Kotlin** [9], escolhidos devido à experiência prévia do grupo com estas tec-

²*Database Management System (DBMS)* é um software concebido para gerir e organizar dados de forma estruturada. Permite aos utilizadores criar, modificar e consultar bases de dados, bem como gerir a segurança e os controlos de acesso às mesmas.[5].

nologias e à rapidez de desenvolvimento que proporcionam. Adicionalmente, foi utilizada a biblioteca **JDBI** [10] para facilitar o acesso à base de dados **PostgreSQL** e a **API Firestore** [11] para o acesso à base de dados **Firestore**.

Para complementar as tecnologias utilizadas no *backend*, foram integrados serviços externos, como a **API Spoonacular** [12], utilizada para obter dados sobre ingredientes, e a biblioteca **Cloud Storage Kotlin** [13], que permite ao *backend* aceder e interagir com o **Cloud Storage** para o armazenamento eficiente de imagens.

Por fim, a biblioteca **Mockito** [14] foi utilizada nos testes unitários para simular comportamentos de diferentes componentes do sistema.

Capítulo 3

Backend

Neste capítulo o principal objetivo é apresentar a organização do software, a base de dados relacional, o acesso à base de dados, os serviços, a REST API, o sistema de autenticação e o tratamento de erros.

3.1 Organização do Software

O código-fonte do servidor encontra-se na pasta **epicurius/server**. Este código está organizado nas seguintes pastas:

- **config** - que contém as classes responsáveis pela configuração das bases de dados e do pipeline utilizado pela aplicação.
- **http** - que contém as classes utilizadas para processar os pedidos HTTP.
- **services** - que contém as classes que validam os parâmetros e implementam a lógica das operações.
- **repository** - que contém as interfaces e implementações utilizadas para aceder às bases de dados.
- **domain** - que contém todas as entidades principais da aplicação.

3.2 Modelo Relacional

Para armazenar os dados do sistema, como informações dos utilizadores, dados de autenticação, receitas, ingredientes, planos de refeições e outras entidades estruturadas, foi utilizado um sistema de gestão de bases de dados relacionais. O **PostgreSQL** foi a tecnologia escolhida para esse fim, tal como referido na Secção 2.2, devido à sua compatibilidade com consultas relacionais complexas.

Adicionalmente, o sistema recorre à base de dados **Firestore** para armazenar dados que exigem maior capacidade de memória ou que não beneficiam diretamente da estrutura

relacional, como é o caso dos procedimentos detalhados das receitas. Para o armazenamento de imagens, é utilizado o **Cloud Storage**, um serviço otimizado para o armazenamento de ficheiros de grandes dimensões, como, por exemplo, imagens. Esta divisão permite otimizar o desempenho do sistema, aliviando a carga do **PostgreSQL** e garantindo uma melhor performance nas consultas.

3.2.1 User

O modelo apresentado na Figura 3.1 representa as principais entidades do sistema, relacionadas com utilizadores e respetivos dados de autenticação. Esta entidade possui atributos como `name`, `email`, `country`, `intolerancies`, `diets`, etc. É importante referir que o atributo `profile picture`, é armazenado na base de dados **Cloud Storage**, garantindo o desempenho de consultas em **PostgreSQL**, sendo este um ficheiro de elevada dimensão.

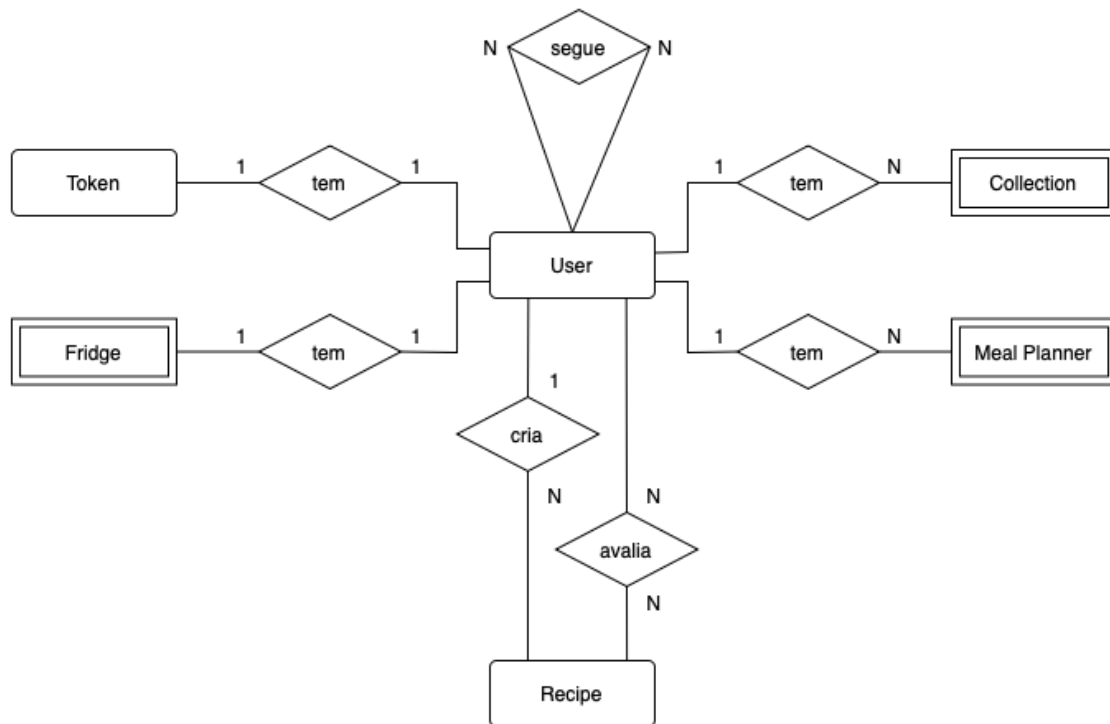


Figura 3.1: Diagrama Entidade-Associação do User

Como apresentado, a Entidade **User** está relacionada com várias outras entidades, tais como a entidade **Token** que armazena a informação encriptada do *token* e a última vez que este *token* foi utilizado pelo utilizador, através do atributo `last used`. Esta informação permite que após 30 dias de inatividade a informação de *token* seja revogada.

De forma a representar os utilizadores que um determinado utilizador segue, a entidade **User** relaciona-se com ela própria, expressando essa informação.

A presente entidade possui uma relação com a entidade fraca **Fridge**, onde estará guardada toda a informação de produtos que o utilizador possua disponíveis, ou seja, atributos como **name**, **quantity**, **open date** e **expiration date**.

A entidade **Recipe** possui dois tipos de relação com a entidade **User**, uma relação em que o utilizador é o autor da receita e uma outra que possibilita a avaliação de uma dada receita pelo utilizador.

Finalmente, existem também as entidades **Collection** e **Meal Planner**. A primeira contém informação sobre as receitas guardadas e publicadas pelo utilizador, distinguidas pelo atributo **type**. Esta entidade possui ainda o atributo **name** referente a uma dada coleção. A segunda entidade consiste no planeamento diário de refeições do utilizador tendo atributos como **meal**, referente ao horário da refeição (pequeno-almoço, almoço, jantar ou lanche), **date** que corresponde à data da planificação e opcionalmente o valor máximo de calorias a atingir, **max calories**.

3.2.2 Token

Como referido na Secção 3.2.1, a entidade **Token** relaciona-se com a entidade **User**, identificando o utilizador no sistema armazenando um *token* encriptado pelo atributo **token** e a data da última utilização no atributo **last used**. Após 30 dias de inutilização por parte do utilizador do *token* que o identifica, o mesmo é revogado.

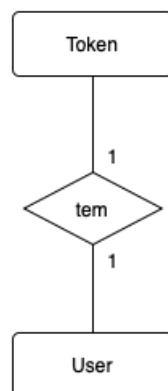


Figura 3.2: Diagrama Entidade-Associação de Token

3.2.3 Recipe

A Figura 3.3 apresenta o modelo com as relações da entidade **Recipe**. Esta entidade representa uma receita culinária e possui diversos atributos que a caracterizam de forma completa, sendo eles um identificador único **id**, **name**, **description**, **instructions**, **preparation time**, **dish types**, **cuisines**, e **pictures**.

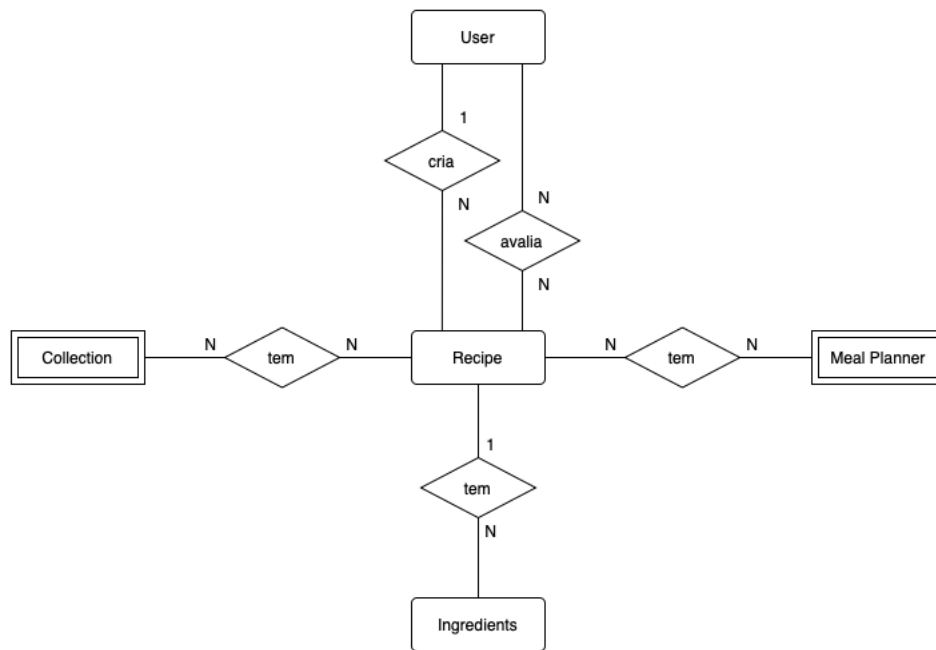


Figura 3.3: Diagrama Entidade-Associação de Recipe

Além disso, a entidade **Recipe** está associada a um grupo de dados nutricionais, **macros**, que inclui os atributos **calories**, **protein**, **carbs** e **fat**. Há também uma relação com restrições alimentares **restrictions**, que podem abranger tanto **intolerances** quanto **diets**.

A entidade **Recipe** possui dados de grandes dimensões, como tal, houve a necessidade de armazenar os atributos **description** e **instructions** no **Firestore**, preservando a integridade do **PostgreSQL**. A mesma lógica foi aplicada para o atributo **pictures**, no entanto, este é armazenado no **Cloud Storage**.

Tal como referido na Secção 3.2.1, **Recipe** relaciona-se com a entidade **User** podendo ser criada ou avaliada por um determinado utilizador.

A relação entre a entidade **Recipe** e a entidade **Ingredients** representa os ingredientes necessários a uma determinada receita. A entidade **Ingredients** possui atributos como o nome do ingrediente **name**, **quantity** e a unidade necessária do determinado ingrediente **unit**.

Por último, a entidade **Recipe** relaciona-se com as entidades **Collection** e **Meal Planner**, relações nas quais as receitas fazem parte de coleções ou da planificação estabelecida, respetivamente.

3.2.4 Ingredients

A entidade apresentada na Figura 3.4, **Ingredients** relaciona-se apenas com a entidade **Recipe** que armazena toda a informação referente aos ingredientes necessários à realização de uma determinada receita, nomeadamente atributos como nome do ingrediente **name**,

quantity e unidade de medida que deverá ser utilizada, **unit**, assim como é referido na Secção 3.2.4.

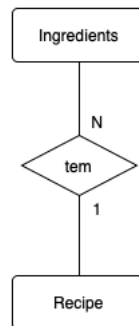


Figura 3.4: Diagrama Entidade-Associação de Ingredients

3.2.5 Collection

Na Figura 3.5 é possível observar o modelo da entidade **Collection**, responsável por armazenar as receitas que o utilizador guardou como *Favourites* e a organização definida pelo utilizador responsável pela criação das receitas no seu perfil, com a designação *Kitchen Book*. A entidade é caracterizada pelos atributos **name**, com o nome da coleção e **type** que distingue a designação da coleção, podendo ser *Favourites* ou *Kitchen Book*, como referido anteriormente.

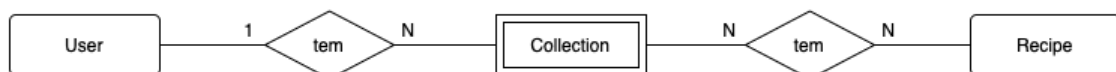


Figura 3.5: Diagrama Entidade-Associação da Collection

A presente entidade possui apenas duas relações, uma com a entidade **User** que reflete o utilizador proprietário de uma determinada coleção, e outra com a entidade **Recipe** correspondente à composição de uma coleção com uma determinada receita.

3.2.6 Fridge

O modelo da Figura 3.6 representa a entidade **Fridge**, com o propósito de armazenar a informação de produtos guardados por um determinado utilizador, para geri-los. O grupo de dados associado à entidade **Fridge**, **product**, inclui os atributos **entry number**, identificador único associado à entrada de um produto, **name**, **quantity**, e datas de abertura e expiração do produto introduzido, respetivamente, **open date** e **expiration date**.

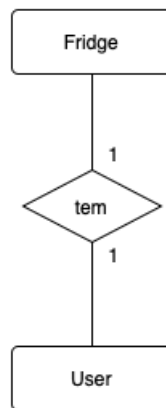


Figura 3.6: Diagrama Entidade-Associação de Fridge

3.2.7 MealPlanner

A Figura 3.7 apresenta o modelo da entidade **Meal Planner**. Esta entidade é caracterizada pelos atributos **date**, correspondente à data da planificação e **max calories**, um atributo opcional para definir a meta de calorias diárias a atingir.

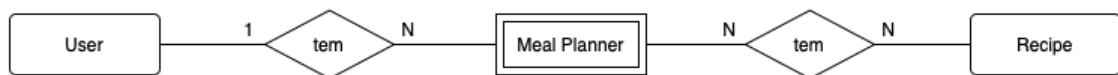


Figura 3.7: Diagrama Entidade-Associação de Meal Planner

A entidade **Meal Planner** possui duas relações, uma com a entidade **User**, demonstrado a que utilizador a planificação pertence, e outra com a entidade **Recipe** associando receitas a variadas planificações. Esta última relação possui um atributo, **meal time**, que associa a receita de uma planificação a uma refeição, ou seja, pequeno-almoço, almoço, jantar e lanche.

Para um panorama mais geral, ver Figura A.1.

3.3 Modelo Físico

3.3.1 Aspetos e Restrições de Design

User

Os atributos **intolerances** e **diets** da entidade **User** são array que admitem valores correspondentes a índices dos enumerados *Intolerance* e *Diet*, os quais possuem termos aceites dentro do contexto da aplicação.

Recipe

A lógica explicada no início da Sub-Secção 3.3.1, também se aplica para a entidade **Recipe** a qual possui os atributos **intolerances** e **diets**.

Followers

A entidade **Followers** é originária da relação **N:N** da entidade **User** como ela própria. De forma a não permitir que um dado utilizador se siga a si mesmo, na entidade **Followers**, foi adicionada uma restrição para que o **follower id** não seja igual ao **user id**, ou seja, o utilizador a seguir e seguido não pode ser o mesmo.

Recipe

Atributos com valores inteiros da entidade **Recipe** têm de ser positivos, nomeadamente **servings**, **preparation time**, **meal type**. Este atributo possui ainda outra restrição, tem de estar dentro do intervalo 0 a 12, assim como o atributo **cuisine**, no entanto para este caso o intervalo altera-se para 0 a 25. Os atributos **meal type** e **cuisine** representam índices de enumerados, *Meal Type* e *Cuisine* respetivamente, semelhante à lógica explicada para as entidades **User** e **Recipe** na Sub-Secção 3.3.1. Todos os atributos associados a valores nutricionais, se atribuídos valor, possuem a constrição de serem positivos, sendo eles **calories**, **protein**, **fat** and **carbs**.

A entidade detém ainda o atributo **pictures names** com restrições. Tendo em consideração que o presente atributo é um *array*, o tamanho do mesmo tem de estar compreendido entre 1 e 3, ou seja, uma receita deverá ter a ela associada pelo menos 1 foto, e no máximo 3.

Recipe Rating

Para garantir que o autor de uma receita não pode avaliar a sua própria receita, o atributo **user id**, da entidade **Recipe Rating**, terá de ser diferente do atributo **author id**, da entidade **Recipe**, possibilitando tal operação.

Esta entidade possui ainda outra restrição no atributo **rating**, na qual o valor inteiro associado terá de estar compreendido no intervalo de 1 a 5.

Collection

Na entidade **Collection**, o atributo **type** poderá apenas assumir o valor 0 ou 1, uma vez que estes valores representam o índice de um enumerado com os tipos da coleção, sendo eles **Favourites** e **Kitchen Book**, semelhante à lógica da Sub-Secção **Recipe**.

Meal Planner

Para cada planeamento é possível, opcionalmente, definir a meta de calorias a atingir para um determinado dia, por isso o atributo **max calories** da entidade **Meal Planner**, caso seja

atribuído valor, deve ser positivo.

Meal Planner Recipe

Esta entidade obtida através da relação **N:N** entre as entidades **Recipe** e **Meal Planner**, possui um atributo denominado de **meal type**, o qual pode assumir um valor pertencente a um intervalo de 0 a 3. Neste atributo a lógica das Sub-Secções **Recipe** e **Collection**, é aplicada mas neste caso para o enumerado *Meal Type*.

Na Figura A.2, é possível ter uma observação detalhada do modelo.

3.4 Repositório

A camada dos repositórios contém as interfaces e implementações utilizadas para aceder à base de dados **PostgreSQL**, recorrendo à biblioteca **JDBI** [10], à base de dados **Firestore** através da **API Firestore** [11], ao **Cloud Storage** utilizando a **Cloud Storage Kotlin Library** [13], à **Cloud Function** e à **Spoonacular API** através de um cliente **HTTP**.

3.4.1 Acessos

PostgreSQL

Para permitir a interação com a base de dados **PostgreSQL**, foram criadas duas interfaces: **Transaction** e **TransactionManager**.

Na Listagem 3.1, a interface **Transaction** define o contrato para os vários repositórios de dados que podem ser acedidos, como, por exemplo, os repositórios de utilizadores, receitas, entre outros.

A interface **TransactionManager**, apresentada na Listagem 3.2, define o método para a execução de blocos de código dentro de um contexto de transação. Este método designado de **run** é implementado na classe **JdbiTransactionManager**. O método **run** executa um bloco de código utilizando o nível de isolamento de transação por defeito da base de dados, que neste caso é *Read-committed* para o **PostgreSQL**.

```
1 interface Transaction {  
2     val userRepository: UserRepository  
3     val recipeRepository: RecipeRepository  
4     ...  
5 }
```

Listagem 3.1: Transaction

```
1 interface TransactionManager {  
2     fun <R> run(block: (Transaction) -> R): R  
3 }
```

Listagem 3.2: Transaction Manager

As interfaces **Transaction** e **TransactionManager** foram implementadas pelas classes **JdbiTransaction** e **JdbiTransactionManager**, respetivamente, seguindo os princípios aprendidos na unidade curricular de Desenvolvimento de Aplicações Web (DAW), para a interação com bases de dados utilizando a biblioteca **JDBI**. Para além destas classes, foram criadas várias classes **mapper** para converter tipos específicos da base de dados em tipos ou objetos **Kotlin** que não podem ser convertidos automaticamente.

A classe **JdbiTokenRepository**, apresentada na Listagem 3.3, implementa a interface **TokenRepository** e é responsável por interagir com a base de dados para executar essas operações. Utiliza o objeto **Handle**, fornecido pela biblioteca **JDBI**, para executar instruções **SQL** e associar os parâmetros necessários.

```

1 class JdbiTokenRepository(private val handle: Handle) : TokenRepository {
2
3     override fun createToken(tokenHash: String, lastUsed: LocalDate, userId: Int) {
4         handle.createUpdate(
5             """
6                 INSERT INTO dbo.token(hash, last_used, user_id)
7                 VALUES (:token_hash, :last_used, :userId)
8             """
9         )
10        .bind("token_hash", tokenHash)
11        .bind("last_used", lastUsed)
12        .bind("userId", userId)
13        .execute()
14    }
15
16    override fun deleteToken(userId: Int) {
17        handle.createUpdate(
18            """
19                DELETE FROM dbo.token
20                WHERE user_id = :userId
21            """
22        )
23        .bind("userId", userId)
24        .execute()
25    }
26 }

```

Listagem 3.3: Jdbi Token Repository

Firestore

À semelhança do que foi feito para o acesso à base de dados **PostgreSQL** com a classe **JdbiTransactionManager**, criou-se a classe **FirestoreManager** para gerir o acesso ao **Firestore**. Contudo, como o **Firestore** não necessita de um método **run** para executar blocos de código dentro de uma transação, esta classe é mais simples, limitando-se a expor os repositórios necessários para a interação com a base de dados **Firestore**, como exemplificado na Listagem 3.4.

```

1 @Repository
2 class FirestoreManager(firestore: Firestore) {
3     val recipeRepository = FirestoreRecipeRepository(firestore)
4 }

```

Listagem 3.4: Firestore Transaction Manager

Cloud Storage

À semelhança do que foi implementado para o acesso ao Firestore com a classe `FirestoreManager`, a gestão do acesso ao Cloud Storage também foi encapsulada numa classe dedicada, a `CloudStorageManager`.

A classe `CloudStorage` apresentada na Listagem 3.5 abstrai a interação com a API do Google Cloud Storage, fornecendo métodos para obter blobs (objetos armazenados), criar identificadores para esses blobs e definir informações associadas ao seu conteúdo.

```
1 class CloudStorage(val storage: Storage, private val bucketName: String) {
2     fun getBlob(objectName: String): Blob = storage.get(bucketName, objectName)
3
4     fun createBlobId(objectName: String): BlobId = BlobId.of(bucketName, objectName)
5
6     fun createBlobInfo(blobId: BlobId, contentType: String): BlobInfo =
7         BlobInfo.newBuilder(blobId).setContentType(contentType).build()
8 }
```

Listagem 3.5: Cloud Storage

Cloud Function e API Spoonacular

À semelhança do que foi feito para o Firestore e o Cloud Storage, foram criadas as classes `CloudFunctionManager` e `SpoonacularManager` para gerir a interação com a Cloud Function e a API externa Spoonacular, respetivamente. Ambas utilizam um cliente HTTP configurado para realizar chamadas assíncronas aos *endpoints* de ambos os serviços.

O `HttpClientConfigurer` apresentado na Listagem 3.6 configura um cliente HTTP assíncrono que suporta métodos GET e POST, facilitando a comunicação eficiente com serviços externos.

```
1 @Configuration
2 class HttpClientConfigurer : HttpClientConfig {
3     val httpClient: HttpClient = HttpClient.newHttpClient()
4
5     override suspend fun get(uri: String): String =
6         httpClient.sendAsync(
7             HttpRequest.newBuilder(URI(uri))
8                 .GET()
9                 .build(),
10            HttpResponse.BodyHandlers.ofString()
11        ).await().body()
12
13     override suspend fun post(uri: String, body: Map<String, String>): String =
14         httpClient.sendAsync(
15             HttpRequest.newBuilder(URI(uri))
16                 .header("Content-Type", "application/json")
17                 .POST(HttpRequest.BodyPublishers.ofString(Json.encodeToString(body)))
18                 .build(),
19            HttpResponse.BodyHandlers.ofString()
20        ).await().body()
21 }
```

Listagem 3.6: Http Client Configurer

3.5 Serviços

A camada de serviços é responsável por implementar a lógica de negócio da aplicação. As classes desta camada recebem os parâmetros dos controladores, validam-nos e executam as operações necessárias, invocando os métodos apropriados dos repositórios para aceder à base de dados. Entre as funcionalidades tratadas nesta camada encontram-se ações como a criação e autenticação de utilizadores, e, criação de receitas, coleções, entre outras operações.

Algumas destas validações são realizadas logo no momento da desserialização dos dados recebidos no corpo do pedido, recorrendo a anotações da biblioteca **Jakarta Validation**[15]. Esta biblioteca permite validar de forma simples e eficaz se, por exemplo, um e-mail está num formato válido, se uma palavra-passe cumpre os critérios definidos ou se um nome respeita os limites de tamanho.

Estas validações são aplicadas automaticamente durante o processo de desserialização dos dados, não sendo necessário repetir as verificações na camada de serviços.

Para criar os serviços, foi utilizada a anotação **@Service** do **Spring**. Esta anotação permite a criação de um serviço que é gerido e instanciado automaticamente, recorrendo à reflexão. Além disso, possibilita a injeção das dependências necessárias ao serviço, como o **TransactionManager** para a interação com a base de dados **PostgreSQL**, conforme ilustrado na Listagem 3.7.

```
1 @Service
2 class UserService(
3     private val tm: TransactionManager,
4     private val cs: CloudStorageManager,
5     private val userDomain: UserDomain,
6     private val pictureDomain: PictureDomain,
7     private val countriesDomain: CountriesDomain
8 ) { ... }
```

Listagem 3.7: Exemplo de uma classe dos Serviços

No exemplo apresentado na Listagem 3.7, o serviço **UserService** possui cinco dependências: **TransactionManager**, **CloudStorageManager**, **UserDomain**, **PictureDomain** e **CountriesDomain**. Estas são injetadas automaticamente pelo **Spring** e permitem que o serviço execute as operações necessárias com o apoio de outros componentes da aplicação.

Na Listagem 3.8, é apresentado um exemplo de um método do serviço **UserService**, que demonstra o processo de criação de um novo utilizador no sistema. Nesta função, são realizadas várias validações, como a verificação da existência de utilizadores com o mesmo nome ou email, a validade do código do país e a correspondência entre as palavras-passe. Após as validações, a palavra-passe é encriptada e o novo utilizador é criado na base de dados, utilizando o **TransactionManager** para garantir que a operação decorre dentro de uma transação.

A camada de serviços é responsável por aplicar estas regras de negócio e garantir que os dados se encontram num formato adequado antes de serem persistidos na base de dados.

```
1 fun createUser(name: String, email: String, country: String, password: String,  
   confirmPassword: String): String {  
2     if (checkIfUserExists(name, email) != null) throw UserAlreadyExists()  
3     if (!countriesDomain.checkIfCountryCodeIsValid(country)) throw InvalidCountry()  
4     checkIfPasswordsMatch(password, confirmPassword)  
5     val passwordHash = userDomain.encodePassword(password)  
6     val userId = tm.run { it.userRepository.createUser(name, email, country, passwordHash) }  
7     return createToken(userId)  
8 }
```

Listagem 3.8: Exemplo de uma função do serviço UserService

3.6 HTTP

3.6.1 Gestão de Endpoints e Routing

O *framework* **Spring Boot** utiliza uma abordagem baseada em anotações para definir os *endpoints* da aplicação. A anotação `@RestController` designa uma classe como controlador, responsável por receber pedidos HTTP e encaminhá-los para métodos que processam esses pedidos. Adicionalmente, a anotação `@RequestMapping` estabelece o caminho base para todos os *endpoints* dentro da classe. As anotações `@GetMapping`, `@PostMapping`, entre outras, especificam os métodos HTTP que os endpoints irão tratar, conforme demonstrado na Listagem 3.9.

```
1 @RestController
2 @RequestMapping(Uris.PREFIX) // /api
3 class MenuController(private val menuService: MenuService) {
4
5     @GetMapping(Uris.Menu.MENU) // /api/menu
6     fun getDailyMenu(...): ResponseEntity<*> {...}
7 }
```

Listagem 3.9: Exemplo de routing

Na Listagem 3.9, a classe `MenuController` é definida como um controlador responsável por receber pedidos HTTP que comecem com o prefixo `/api`. Adicionalmente, o método `getDailyMenu` está configurado para tratar pedidos `GET` feitos ao *endpoint* `/api/menu`.

Nos métodos que necessitam de um parâmetro na sua *URI*, é possível definir esse parâmetro nas anotações dos métodos HTTP. Esse parâmetro é depois passado como argumento, com o mesmo nome definido na anotação, para o método que irá processar o pedido, conforme ilustrado na Listagem 3.10.

```
1 @RestController
2 @RequestMapping(Uris.PREFIX) // /api
3 class RecipeController(private val recipeService: RecipeService) {
4
5     @GetMapping(Uris.Recipe.RECIPE) // /recipes/{id}
6     suspend fun getRecipe(
7         authenticatedUser: AuthenticatedUser,
8         @PathVariable id: Int,
9     ): ResponseEntity<*> {
10         val recipe = recipeService.getRecipe(id, authenticatedUser.user.id)
11         return ResponseEntity
12             .ok()
13             .body(GetRecipeOutputModel(recipe))
14     }
```

Listagem 3.10: Exemplo de routing com um parâmetro na *path*

Se o cliente fizer um pedido **HTTP GET** ao *endpoint* `/api/recipes/1`, o pedido será encaminhado para o método `getRecipe` da classe `RecipeController`. Este método irá processar o pedido com o parâmetro `id` igual a 1 e devolverá uma resposta ao cliente.

3.6.2 Desserialização do Corpo do Pedido

Para desserializar os dados recebidos no corpo do pedido, é utilizada a anotação `@RequestBody`. Esta anotação é responsável por converter os dados do corpo do pedido num objeto do tipo definido como parâmetro do método, conforme ilustrado na Listagem 3.13.

Para desserializar os dados recebidos no corpo do pedido, é utilizada a anotação `@RequestBody`. Esta anotação é responsável por converter os dados do corpo do pedido num objeto do tipo definido como parâmetro do método. Para além disso, a anotação `@Valid` é usada em conjunto para ativar as validações definidas no modelo de dados (por exemplo, na Listagem ??). Esta validação é feita automaticamente com base nas anotações presentes nos campos do modelo, como `@NotBlank`, `@Email`, `@Size`, entre outras, pertencentes à biblioteca `Jakarta Validation`. A Listagem 3.11 ilustra um exemplo de um método onde estas anotações são utilizadas no contexto de um pedido de registo de utilizador.

```
1 @RestController
2 @RequestMapping("/api")
3 class UserController() {
4     @PostMapping(Uriis.User.SIGNUP)
5     fun signUp(
6         @Valid @RequestBody body: SignUpInputModel,
7         response: HttpServletResponse
8     ): ResponseEntity<*> {...}
9 }
```

Listagem 3.11: Exemplo de um método com corpo no pedido

Na Listagem 3.11, o método `signUp` do `UserController` recebe um objeto `SignUpInputModel` como parâmetro, que é automaticamente desserializado a partir dos dados enviados no corpo do pedido. Além disso, a anotação `@Valid` é utilizada para validar o conteúdo recebido, garantindo que os dados cumprem os critérios definidos na classe do modelo.

3.7 Especificação OpenAPI

As rotas da API estão organizadas por módulos de funcionalidade, facilitando a navegação e a compreensão da estrutura do sistema.

As tabelas apresentadas nas próximas secções descrevem as rotas disponíveis para cada grupo, especificando o método HTTP, o caminho da rota, uma breve descrição da funcionalidade e se é necessária autenticação para o seu acesso.

Tabela 3.1: Rotas da API de autenticação

Método	Rota	Descrição	Autenticação
POST	/api/signup	Regista um novo utilizador no sistema	Não
POST	/api/login	Autentica um utilizador	Não
POST	/api/logout	Termina a sessão do utilizador	Sim

Tabela 3.2: Rotas da API relacionadas com o utilizador

Método	Rota	Descrição	Autenticação
GET	/api/users	Pesquisar utilizadores	Sim
GET	/api/users/{username}	Obter o perfil de um utilizador	Sim
GET	/api/user	Obter o utilizador autenticado	Sim
GET	/api/user/intolerances	Obter as intolerâncias do utilizador	Sim
GET	/api/user/diets	Obter as dietas do utilizador	Sim
GET	/api/user/follow-requests	Obter os pedidos de <i>following</i> do utilizador	Sim
GET	/api/user/followers	Obter os seguidores do utilizador	Sim
GET	/api/user/following	Obter os utilizadores seguidos pelo utilizador	Sim
GET	/api/user/feed	Obter o feed do utilizador	Sim
PATCH	/api/user	Atualizar os dados do utilizador	Sim
PATCH	/api/user/picture	Atualizar a fotografia de perfil do utilizador	Sim
PATCH	/api/user/follow/{username}	Seguir um utilizador	Sim
PATCH	/api/user/follow-requests/{username}	Aceitar, rejeitar ou cancelar um pedido de <i>following</i>	Sim
PATCH	/api/user/password	Repor a palavra-passe do utilizador	Não
DELETE	/api/user	Eliminar o utilizador	Sim
DELETE	/api/user/follow/{username}	Deixar de seguir um utilizador	Sim

Tabela 3.3: Rotas da API relacionadas com o frigorífico do utilizador

Método	Rota	Descrição	Autenticação
GET	/api/fridge	Obter o frigorífico do utilizador	Sim
POST	/api/fridge	Adicionar um produto ao frigorífico	Sim
PATCH	/api/fridge/product/{entryNumber}	Atualizar um produto no frigorífico	Sim
DELETE	/api/fridge/product/{entryNumber}	Remover um produto do frigorífico	Sim

Tabela 3.4: Rotas da API relacionadas com receitas

Método	Rota	Descrição	Autenticação
GET	/api/recipes	Obter uma receita	Sim
GET	/api/recipes/{id}	Pesquisar receitas	Sim
GET	/api/recipes/{id}/rate	Obter a avaliação de uma receita	Sim
GET	/api/recipes/{id}/rate/self	Obter a avaliação do utilizador para uma receita	Sim
POST	/api/recipes/{id}/rate	Avaliar uma receita	Sim
POST	/api/recipes	Criar uma receita	Sim
PATCH	/api/recipe	Atualizar uma receita	Sim
PATCH	/api/recipes/{id}/rate	Atualizar uma avaliação dada a uma receita	Sim
PATCH	/api/recipes/{id}/pictures	Atualizar imagens de uma receita	Sim
DELETE	/api/recipes/{id}	Eliminar uma receita	Sim
DELETE	/api/recipes/{id}/rate	Eliminar uma avaliação de uma receita	Sim

Tabela 3.5: Rotas da API relacionadas com ingredientes

Método	Rota	Descrição	Autenticação
GET	/api/ingredients	Obter uma lista de ingredientes da API Spoonacular	Sim
GET	/api/ingredients/substitutes	Obter uma lista de ingredientes substitutos da API Spoonacular	Sim
POST	/api/ingredients	Identificar ingredientes presentes numa imagem	Sim

Tabela 3.6: Rotas da API relacionadas com coleções

Método	Rota	Descrição	Autenticação
GET	/api/collections/{id}	Obter uma coleção	Sim
POST	/api/collections/{id}	Criar uma coleção	Sim
POST	/api/collections/{id}/recipes	Adicionar uma receita a uma coleção	Sim
PATCH	/api/collections/{id}	Atualizar uma coleção	Sim
DELETE	/api/collections/{id}/recipes/{id}	Remover uma receita de uma coleção	Sim
DELETE	/api/collections/{id}	Eliminar uma coleção	Sim

Tabela 3.7: Rotas da API relacionadas com o planeamento de refeições

Método	Rota	Descrição	Autenticação
GET	/api/planner	Obter o planeamento semanal de refeições	Sim
GET	/api/planner/{date}	Obter o planeamento diário de refeições	Sim
POST	/api/planner	Criar um planeamento diário de refeições	Sim
POST	/api/planner/{date}	Adicionar uma receita ao planeamento diário	Sim
PATCH	/api/planner/{date}	Atualizar o planeamento diário de refeições	Sim
PATCH	/api/planner/{date}/calories	Atualizar o máximo de calorias do planeamento diário	Sim
DELETE	/api/planner/{date}/{mealTime}	Remover uma receita do planeamento diário	Sim
DELETE	/api/planner/{date}	Eliminar o planeamento diário de refeições	Sim

Tabela 3.8: Rotas da API relacionadas com o menu diário

Método	Rota	Descrição	Autenticação
GET	/api/menu	Obter um menu diário	Sim

3.8 Autenticação

A aplicação utiliza um sistema de autenticação baseado em **email** ou **nome de utilizador** e **palavra-passe**. Após o *login*, o servidor gera um único **token de sessão**, que é utilizado para autenticar o utilizador em todos os pedidos seguintes, simplificando assim o processo de autenticação.

Para garantir a segurança e manter o sistema eficiente, foi implementado um mecanismo que invalida automaticamente *tokens* que não são utilizados há mais de 30 dias. Desta forma, previne-se a reutilização indevida de *tokens* antigos e garante-se que apenas sessões ativas continuam válidas.

3.8.1 Geração do Token

O sistema para a geração de *tokens* utiliza um gerador baseado em `SecureRandom`[16], que permite criar um *token* seguro e imprevisível com base numa sequência de bytes aleatórios, codificados posteriormente em **Base64**.

Após a sua criação, o *token* é imediatamente convertido num *hash*, de modo a nunca ser armazenado em texto simples na base de dados. Esta operação é feita utilizando a biblioteca `java.security`[17] através da classe `MessageDigest`[18], com o algoritmo `SHA-256`¹, um dos algoritmos de *hash* mais utilizados e seguros da atualidade. O valor resultante é também codificado em **Base64**, garantindo a sua integridade e compatibilidade com diferentes formatos de armazenamento e transmissão.

A seguir apresenta-se a implementação deste mecanismo.

```
1 fun generateTokenValue(): String =  
2     ByteArray(TOKEN_SIZE_IN_BYTES).let { byteArray ->  
3         SecureRandom.getInstanceStrong().nextBytes(byteArray)  
4         Base64.getEncoder().encodeToString(byteArray)  
5     }  
6  
7 fun hashToken(token: String): String = tokenEncoder.hash(token)
```

Listagem 3.12: Criação e codificação do Token

¹”Secure Hash Algorithm 256-bit é uma função hash criptográfica que converte qualquer entrada de dados em uma string de 256 bits (32 bytes). Esta string é única e não pode ser revertida para o conteúdo original. É amplamente usado em segurança como em assinaturas digitais e armazenamento de senhas.”

```

1 class Sha256TokenEncoder : TokenEncoder {
2
3     override fun matches(token: String, validationInfo: String) = hash(token) ==
        validationInfo
4
5     override fun hash(input: String): String {
6         val messageDigest = MessageDigest.getInstance("SHA256")
7         return Base64.getEncoder().encodeToString(
8             messageDigest.digest(
9                 Charsets.UTF_8.encode(input).array()
10            )
11        )
12    }
13 }

```

Listagem 3.13: Sha256 Token Encoder

3.8.2 Gestão do Ciclo de Vida do Token

A gestão do ciclo de vida do *token* é feita através de uma tarefa agendada que remove diariamente da base de dados os *tokens* que não são utilizados há mais de 30 dias, garantindo que apenas sessões ativas se mantêm válidas.

Esta tarefa é realizada por uma *scheduled task*, executada diariamente à meia-noite, que remove da base de dados todos os *tokens* cuja última utilização ultrapasse esse limite. A implementação deste mecanismo é ilustrada no código seguinte.

```

1 @Component
2 class TokenScheduler(private val jdbci: Jdbi) {
3
4     @Scheduled(cron = "0 0 0 * * *")
5     fun scheduleDeleteToken() {
6         jdbci.inTransaction<Unit, Exception> { handle ->
7             handle.createUpdate(
8                 """
9                 DELETE FROM dbo.token
10                WHERE current_date - last_used > 30
11                """
12            ).execute()
13        }
14    }
15 }

```

Listagem 3.14: Token Scheduler

3.8.3 Fluxo

Para cada pedido feito pelo cliente ao servidor, o cliente deve incluir o *token* no cabeçalho da requisição, sob o cabeçalho **Authorization**, precedido pela palavra "Bearer". Ao receber o pedido, este é intercetado pela classe **AuthInterceptor**, que verifica se o método solicitado requer autenticação. Se a autenticação for necessária, a classe valida o *token* e determina se este pertence a um utilizador autenticado do sistema.

```

1  @Component
2  class AuthenticationInterceptor(private val requestTokenProcessor: RequestTokenProcessor) :
    HandlerInterceptor {
3
4      override fun preHandle(request: HttpServletRequest, response: HttpServletResponse,
        handler: Any): Boolean {
5          if (handler is HandlerMethod && handler.hasParameterType<AuthenticatedUser>()) {
6              val token = getToken(request) ?: throw MissingUserToken()
7              val authenticatedUser = requestTokenProcessor.getAuthenticatedUser(token)
8              return if (authenticatedUser == null) {
9                  throw AuthenticatedUserNotFound()
10             } else {
11                 AuthenticatedUserArgumentResolver.addSession(authenticatedUser, request)
12                 true
13             }
14         }
15         return true
16     }
17
18     ...
19
20 }

```

Listagem 3.15: Authentication Interceptor

Quando um utilizador realiza um pedido ao servidor, esse pedido é intercetado pela classe `AuthenticationInterceptor`. Esta classe verifica, em primeiro lugar, se o `handler` do pedido é do tipo `HandlerMethod`. Caso seja, avalia se o método possui um parâmetro do tipo `AuthenticatedUser`, o que indica que autenticação é necessária para o processamento desse pedido.

Se for detetado que o método requer autenticação, a função `getToken` tenta extrair o ***token de sessão*** a partir dos *headers* ou dos *cookies* da requisição. Se nenhum *token* for encontrado, é lançada uma exceção `MissingUserToken`. Caso o *token* esteja presente, é invocada a classe `RequestTokenProcessor`, responsável por validar o *token* e devolver a informação do utilizador autenticado. Esta validação consiste em verificar se o *token* fornecido corresponde a um utilizador existente e ainda válido, retornando, em caso de sucesso, um objeto `AuthenticatedUser` que inclui tanto os dados do utilizador como o próprio *token* autenticado. Se o *token* não corresponder a um utilizador autenticado, é lançada a exceção `AuthenticatedUserNotFound`. Por outro lado, se a autenticação for bem-sucedida, a sessão do utilizador é associada ao pedido através da classe `AuthenticatedUserArgumentResolver`, permitindo o acesso à informação do utilizador autenticado no restante ciclo do pedido.

Este mecanismo garante que apenas utilizadores autenticados conseguem aceder aos métodos que o requerem, reforçando a segurança e a integridade da aplicação.

3.8.4 Integração com Cookies

Para aumentar a versatilidade da API e facilitar a integração com diferentes aplicações *front-end*, como um *website*, foi implementado o suporte ao uso de ***cookies*** para armazenar o

token de autenticação.

No lado do servidor, foram criadas funções responsáveis por adicionar o *cookie* ao cliente durante a autenticação e removê-lo quando a sessão termina, assegurando o controlo adequado do ciclo de vida do *token*. Os *cookies* são configurados com as flags `HttpOnly` e `Secure`, garantindo que apenas o servidor tem acesso a estes *cookies* e que são transmitidos exclusivamente através de conexões HTTPS, aumentando a segurança contra ataques comuns como o *Cross-Site Scripting*².

```
1 const val TOKEN = "token"
2
3 fun <T> ResponseEntity<T>.addCookie(response: HttpServletResponse, value: String):
4     ResponseEntity<T> {
5     response.addCookie(
6         Cookie(TOKEN, value).apply {
7             isHttpOnly = true
8             secure = true
9         }
10    )
11    return this
12 }
13 fun <T> ResponseEntity<T>.removeCookie(response: HttpServletResponse): ResponseEntity<T> {
14     response.addCookie(
15         Cookie(TOKEN, "")
16         .apply {
17             isHttpOnly = true
18             secure = true
19             maxAge = 0
20         }
21    )
22    return this
23 }
```

Listagem 3.16: Funções para adicionar e remover cookies de autenticação

²*Cross-Site Scripting* (XSS) é uma vulnerabilidade de segurança comum em aplicações web, que permite a um atacante injetar scripts maliciosos em páginas visualizadas por outros utilizadores, podendo roubar dados sensíveis ou executar ações em nome das vítimas.

3.9 Tratamento de erros

O sistema de tratamento de erros foi concebido para devolver ao cliente uma resposta com um *status code* e uma mensagem específicos, com base no erro lançado pela aplicação. Este sistema utiliza a anotação `@ControllerAdvice`, que permite a criação de uma classe responsável por intercepar exceções lançadas pela aplicação. Essa classe devolve então uma resposta ao cliente com o *status code* e a mensagem apropriados.

```
1 @ControllerAdvice
2 class ExceptionHandler {
3
4     @ExceptionHandler(
5         value = [
6             IllegalArgumentException::class,
7             IllegalStateException::class,
8             InvalidCountry::class,
9             IncorrectPassword::class,
10            ...
11        ]
12    )
13    fun handleBadRequest(request: HttpServletRequest, ex: Exception) =
14        ex.handle(
15            request = request,
16            status = HttpStatus.BAD_REQUEST
17        )
18
19    ...
20
21    companion object {
22        ...
23
24        private fun Exception.handle(
25            request: HttpServletRequest,
26            status: HttpStatus,
27            type: String = toProblemType(),
28            title: String = getName(),
29            detail: String? = message,
30            headers: HttpHeaders? = null
31        ): ResponseEntity<Problem> =
32            Problem(
33                type = URI.create(PROBLEMS_DOCS_URI + type),
34                title = title,
35                detail = detail,
36                instance = URI.create(request.requestURI)
37            ).toResponse(status, headers)
38    }
39 }
```

Listagem 3.17: Exemplo de um Exception Handler

O método `handleBadRequest` é definido para intercepar exceções dos tipos `IllegalArgumentException` e `IllegalStateException`, bem como exceções específicas da aplicação que representem erros do cliente e sejam adequadas para o *status code* HTTP 400 Bad Request, como, por exemplo, a exceção `InvalidCountry`. Este método devolve uma resposta ao cliente sob a forma de um objeto `Problem`, que representa um problema ocorrido na aplicação, incluindo o respetivo *status code* e mensagem. Esta resposta consiste num objeto com os campos: *type*,

title, *detail*, *instance*, o respetivo *status code* e os cabeçalhos HTTP.

- **Type** - Um URI que representa o tipo de problema ocorrido e que remete para a documentação do problema.
- **Title** - Título que resume o problema.
- **Detail** - A mensagem da exceção lançada pela aplicação.
- **Instance** - Caminho da API onde a exceção aconteceu.

Por exemplo, ao tentar criar um utilizador que já existe, é lançada uma exceção do tipo `UserAlreadyExists`, que é então convertida na seguinte resposta:

```
1 {  
2   "type": "https://github.com/vascosta/epicurius/tree/main/docs/problems/user-already-  
3     exists",  
4   "title": "User Already Exists",  
5   "detail": "User already exists",  
6   "instance": "/api/signup"  
}
```

Listagem 3.18: Exemplo de resposta de erro indicando que o utilizador já existe no sistema.

3.10 Cloud Function

A **Cloud Function** fornece suporte ao *backend* do sistema. O seu principal objetivo é interagir com a **API Vision** da **Google** para extrair automaticamente os rótulos presentes numa imagem previamente armazenada no **Cloud Storage**, facilitando a identificação dos ingredientes nela contidos.

O código-fonte correspondente encontra-se na pasta `epicurius/server/cloudFunction`.

3.10.1 Gatilho

A **Cloud Function** é ativada sempre que é feito um pedido HTTP para o seu *endpoint*. Para aumentar a segurança e reduzir pedidos não solicitados, foi restringido o acesso de forma a aceitar apenas pedidos com o método HTTP `POST`. Qualquer tentativa de invocação com um método diferente resulta numa resposta com o código de erro `405 - Method Not Allowed`.

A verificação do método HTTP é realizada no início da execução da função, conforme ilustrado no excerto de código seguinte:

```
1 public class GetIngredientsFromPicture implements HttpFunction {
2
3     ...
4
5     @Override
6     public void service(HttpRequest request, HttpResponse response) throws Exception {
7         if (!request.getMethod().equals("POST")) {
8             response.setStatusCode(405);
9             response.getWriter().write(gson.toJson(Map.of("error", "Method not allowed")));
10            return;
11        }
12
13        ...
14
15    }
16 }
```

Listagem 3.19: Validação do método HTTP na Cloud Function

Ao limitar a função ao método `POST`, cumprem-se as boas práticas de design de APIs REST. Este método é utilizado para operações que envolvem a submissão de dados, como o envio de uma imagem para ser processada, garantindo que o conteúdo não seja exposto nos parâmetros do URL, como aconteceria num pedido `GET`. Além disso, ao aceitar apenas `POST` evita-se que a função seja invocada inadvertidamente por navegadores ou serviços externos que tentem aceder ao *endpoint* com métodos não permitidos.

3.10.2 API Vision

A **Google Cloud Vision API** é um serviço de *machine learning* que permite analisar o conteúdo de imagens, fornecendo informações relevantes como deteção de objetos, rostos, texto, e rótulos que descrevem o que está presente na imagem. Esta API utiliza modelos de

aprendizagem profundamente treinados com grandes volumes de dados visuais, sendo capaz de reconhecer uma vasta gama de elementos com elevada precisão.

No contexto do sistema, a **API Vision** é utilizada para identificar automaticamente ingredientes a partir de fotografias tiradas pelo utilizador. Através da funcionalidade de **label detection**, a API devolve uma lista de rótulos descritivos. Estes rótulos são então enviados para o servidor, que é responsável por filtrar e comparar os resultados com a base de dados de ingredientes da **API Spoonacular**, identificando quais os rótulos que correspondem efetivamente a ingredientes válidos.

Na figura 3.8 apresenta-se um exemplo de imagem submetida à API, seguido da resposta obtida que contém os rótulos detetados:



Figura 3.8: Imagem enviada para a Cloud Function com o objetivo de identificar ingredientes.

```
1 {  
2   [  
3     "red",  
4     "fruit",  
5     "cherry tomato",  
6     "tomato",  
7     "vegetable"  
8   ]  
9 }  
10
```

Listagem 3.20: Possível resposta da Cloud Function com base na imagem enviada

Como se pode observar, a API retorna múltiplos rótulos associados à imagem. Esta abordagem reduz a necessidade de introdução manual de dados, proporcionando uma experiência de utilizador mais simples e eficiente.

Por fim, apesar da utilidade e eficácia da API Vision na identificação de ingredientes a partir de imagens, é importante reconhecer que esta tecnologia não é infalível. Em particular, a precisão dos rótulos pode diminuir quando se tratam de alimentos embalados, onde a visibilidade dos ingredientes é reduzida ou ocorrem interferências visuais (como texto nas embalagens, reflexos ou designs gráficos). Nestes casos, a API pode não identificar corretamente os ingredientes reais presentes na imagem. Por isso, estes rótulos são então enviados para o servidor, que é responsável por filtrar e comparar os resultados com a base de dados de ingredientes da **API Spoonacular**, identificando quais os rótulos que correspondem efetivamente a ingredientes válidos.

3.10.3 Deploy

A **Cloud Function** pode ser implementada diretamente através da interface da **GCP** ou, de forma mais flexível e automatizada, utilizando a ferramenta de linha de comandos `gcloud`. O exemplo seguinte mostra o comando utilizado para o processo de *deploy*:

```
1 gcloud functions deploy get-ingredients-from-picture \
2   --entry-point epicurius.GetIngredientsFromPicture \
3   --runtime java21 \
4   --trigger-http \
5   --allow-unauthenticated \
6   --region=europe-west1 \
7   --gen2
```

Listagem 3.21: Comando para o *deploy* da Cloud Function na GCP

Cada parâmetro deste comando desempenha um papel específico na configuração e *deploy* da função:

- `get-ingredients-from-picture`: Nome atribuído à **Cloud Function**.
- `--entry-point epicurius.GetIngredientsFromPicture`: Define o ponto de entrada da função, ou seja, a classe Java cujo método será executado quando a função for chamada.
- `--runtime java21`: Especifica o ambiente de execução da função. Neste caso, a função é desenvolvida em Java 21.
- `--trigger-http`: Indica que a função será acionada por pedidos HTTP.
- `--allow-unauthenticated`: Permite que a função seja invocada sem necessidade de autenticação.

- `--region=europe-west1`: Define a região de *deploy* da função, com impacto na latência e disponibilidade.

3.11 Testes

Foram realizados testes aos serviços e controladores utilizando a biblioteca **Mockito**[14] para simular o acesso às base de dados. Estes testes foram concebidos para verificar se a lógica nestes módulos estava correta e se os resultados obtidos eram os esperados.

A Listagem 3.22 apresenta um exemplo de teste onde se verifica a funcionalidade de adicionar uma receita a uma coleção. O teste simula várias chamadas ao repositório e valida se a receita foi efetivamente adicionada à coleção correta.

```
1  @Test
2  fun 'Should add a recipe to a collection successfully'() {
3      // given a collection id (FAVOURITE_COLLECTION_ID) and a recipe id (RECIPE_ID)
4
5      // mock
6      val mockJdbiUpdatedCollectionModel = testFavouriteJdbiCollectionModel.copy(
7          recipes = listOf(testJdbiRecipeInfo)
8      )
9      whenever(jdbiCollectionRepositoryMock.getCollectionById(FAVOURITE_COLLECTION_ID))
10         .thenReturn(testFavouriteJdbiCollectionModel)
11      whenever(jdbiCollectionRepositoryMock.checkIfUserIsCollectionOwner(
12          FAVOURITE_COLLECTION_ID, PUBLIC_USER_ID))
13         .thenReturn(true)
14      whenever(jdbiRecipeRepositoryMock.getRecipeById(RECIPE_ID)).thenReturn(
15          testJdbiRecipeModel)
16      whenever(jdbiUserRepositoryMock.checkUserVisibility(testPublicUsername, testPublicUser
17          .id)).thenReturn(true)
18      whenever(jdbiCollectionRepositoryMock.checkIfRecipeInCollection(
19          FAVOURITE_COLLECTION_ID, RECIPE_ID))
20         .thenReturn(false)
21      whenever(jdbiCollectionRepositoryMock.addRecipeToCollection(FAVOURITE_COLLECTION_ID,
22          RECIPE_ID))
23         .thenReturn(mockJdbiUpdatedCollectionModel)
24      whenever(pictureRepositoryMock.getPicture(testJdbiRecipeModel.picturesNames.first(),
25          RECIPES_FOLDER))
26         .thenReturn(byteArrayOf())
27
28      // when adding the recipe to the collection
29      val updatedCollection = addRecipeToCollection(
30          testPublicUser.id, FAVOURITE_COLLECTION_ID, RECIPE_ID
31      )
32
33      // then the recipe is added successfully
34      assertEquals(FAVOURITE_COLLECTION_ID, updatedCollection.id)
35      assertEquals(testRecipeInfo.id, updatedCollection.recipes.first().id)
36  }
```

Listagem 3.22: Exemplo de teste unitário para adição de uma receita a uma coleção

Para testar os restantes módulos da aplicação, incluindo repositórios e a pipeline, foram realizados testes de integração utilizando o **WebTestClient**[19]. Além disso, a aplicação **Postman** foi usada para validar que as respostas do servidor estavam corretas e alinhadas com os resultados esperados.

Capítulo 4

Conclusões

No desenvolvimento deste projeto, foram implementados funcionalidades como registo e autenticação de utilizador e respetivas configurações, *feed* personalizado, criação e publicação de uma receita, pesquisa de receitas, criação de coleções e adição de receitas às mesmas, menu diário, planeamento de refeições e funcionalidade de frigorífico. Apesar de algumas dificuldades enfrentadas, como a deteção imperfeita de alimentos por parte da **API Vision** e envio de imagens em pedidos através do *content-type*, **form/data**.

De uma perspectiva de futuro, as próximas etapas consistem em melhorias de funcionalidades já existentes, finalização da implementação do *Frontend*, e desenvolvimento de requisitos opcionais, como apresentação de sugestões de receitas por etiquetas seguidas pelos utilizadores, recomendação de vinhos para acompanhar refeições principais, adição de vídeos para acompanhamento passo a passo de uma receita, implementação de mini-cursos, em caso de notificação de expiração de um produto, são apresentadas ao utilizador sugestões de receitas para o mesmo, e, finalmente, opção de *download* de receitas, possibilitando o acesso às mesmas na aplicação em modo *offline*.

Concluindo, o projeto **Epicurius** revelou ser um desafio estimulante, no sentido de implementar uma aplicação de raiz e cumprir com planeamento desenvolvido, aplicando todo o conhecimento adquirido durante o curso, e com a oportunidade de explorar novas tecnologias, como é o caso da **API Vision**.

4.1 Requisitos Funcionais Concluídos

No decorrer do projeto foram implementados com sucesso os requisitos funcionais presentes na Secção 1.2, excetuando alguns requisitos.

- **Perfil do Utilizador** - Foi implementado um sistema de autenticação seguro e eficiente;
- **Configurações do Utilizador** - O sistema permite o ajuste de definições pessoais e de privacidade;

- **Feed** - Um feed dinâmico com receitas publicadas pelos utilizadores seguidos, ordenadas cronologicamente com base na data de publicação, foi providenciado;
- **Perfil de Receita** - Possibilidade de obtenção de uma receita com informação relevante, incluindo o autor, tipo de cozinha, tipo de refeição, ingredientes, modo de preparação, entre outros;
- **Publicação de Receitas** - Garantida a possibilidade de criação e publicação de novas receitas por parte dos utilizadores;
- **Pesquisa de Receitas** - A plataforma permite aos utilizadores procurar receitas utilizando:
 1. O nome da receita;
 2. Um formulário de pesquisa com filtros específicos (ex.: tipo de cozinha, tempo de preparação, ingredientes);
 3. Uma imagem captada ou carregada pelo utilizador, analisada pela aplicação para identificar os ingredientes presentes.
- **Coleções** - Cada utilizador pode personalizar a organização das suas receitas publicadas e também poderá guardar como favorito alguma receita publicada na aplicação.
- **Sugestão Diária de um Menu** - O sistema fornece diariamente sugestões de um menu completo que inclui pequeno-almoço, almoço, lanche, jantar, sopa e sobremesa.
- **Planeamento de Refeições** - Funcionalidade que permite ao utilizador organizar refeições para dias específicos da semana, com a possibilidade de definir metas calóricas diárias, já se encontra implementada.
- **Funcionalidade de Frigorífico** - A plataforma permite aos utilizadores adicionar produtos que têm em casa, com indicação de quantidades e datas de validade.

4.2 Próximos Passos

Futuramente, o trabalho a ser desenvolvido engloba melhoria de funcionalidades já implementadas e terminar implementação da componente de **frontend** da aplicação. Importante referir que falta ainda realizar o **deployment** da componente de **backend**.

Adicionalmente, um dos grandes objetivos, após terminados os anteriores, será a implementação de requisitos opcionais.

De seguida, enumera-se com detalhe os requisitos em falta da aplicação:

- **Acompanhamento de uma Receita** - Qualquer utilizador pode acompanhar uma receita passo a passo através de uma interface interativa que apresenta as instruções de

preparação de forma sequencial avançando apenas quando desejar. Esta funcionalidade será implementada ao nível da componente de *frontend* de modo a ser visível e mais simples a sequência de procedimentos;

- **Funcionalidade de frigorífico** - O sistema notifica o utilizador quando um produto se aproxima da data de expiração. Como referido no ponto anterior, esta é também uma funcionalidade ao nível de *frontend*
- **Ligação entre componentes *frontend* e *backend*** - O *frontend* é baseado numa aplicação Android, que já possui alguns *screens* que serão futuramente utilizados, no entanto ainda não possuem *View-Model*, o que impossibilita a sua interação com o *backend*;
- **Deployment do *backend*** - Neste momento a componente *backend* é executada em contentores, por utilização de um *dockerfile*, mas brevemente será feito o seu *deployment*.

Bibliografia

- [1] Mr cook. <https://www.mrcook.app>. Accessed: 2025-05-20.
- [2] Cloud storage. <https://cloud.google.com/storage/docs/introduction>. Accessed: 2025-05-20.
- [3] Cloud functions. <https://cloud.google.com/functions/docs/concepts/overview>. Accessed: 2025-05-20.
- [4] Vision api. <https://cloud.google.com/vision/docs>. Accessed: 2025-05-20.
- [5] Dbms. <https://www.geeksforgeeks.org/introduction-of-dbms-database-management-system-se>. Accessed: 2025-05-20.
- [6] Postgresql. <https://www.postgresql.org/>. Accessed: 2025-05-20.
- [7] Firestore. <https://firebase.google.com/docs/firestore>. Accessed: 2025-05-20.
- [8] Spring mvc. <https://docs.spring.io/spring-framework/reference/web/webmvc.html>. Accessed: 2025-05-20.
- [9] Kotlin. <https://kotlinlang.org/>. Accessed: 2025-05-20.
- [10] Jdbi. <https://jdbi.org/>. Accessed: 2025-05-20.
- [11] Firestore api. <https://cloud.google.com/firestore/docs/reference/rest>. Accessed: 2025-05-20.
- [12] Spoonacular api. <https://spoonacular.com/food-api/docs>. Accessed: 2025-05-20.
- [13] Biblioteca cloud storage kotlin. <https://github.com/googleapis/java-storage?tab=readme-ov-file>. Accessed: 2025-05-20.
- [14] Mockito. <https://github.com/mockito/mockito-kotlin?tab=readme-ov-file>. Accessed: 2025-05-20.
- [15] Java validation. <https://www.baeldung.com/java-validation>. Accessed: 2025-05-20.
- [16] Secure random. <https://docs.oracle.com/javase/8/docs/api/java/security/SecureRandom.html>. Accessed: 2025-05-20.

- [17] Java security. <https://docs.oracle.com/en/java/javase/11/security/java-security-overview1.html>. Accessed: 2025-05-20.
- [18] Message digest. <https://docs.oracle.com/javase/8/docs/api/java/security/MessageDigest.html>. Accessed: 2025-05-20.
- [19] Web test client. <https://docs.spring.io/spring-framework/reference/testing/webtestclient.html>. Accessed: 2025-05-20.

Apêndice A

Modelos de Dados

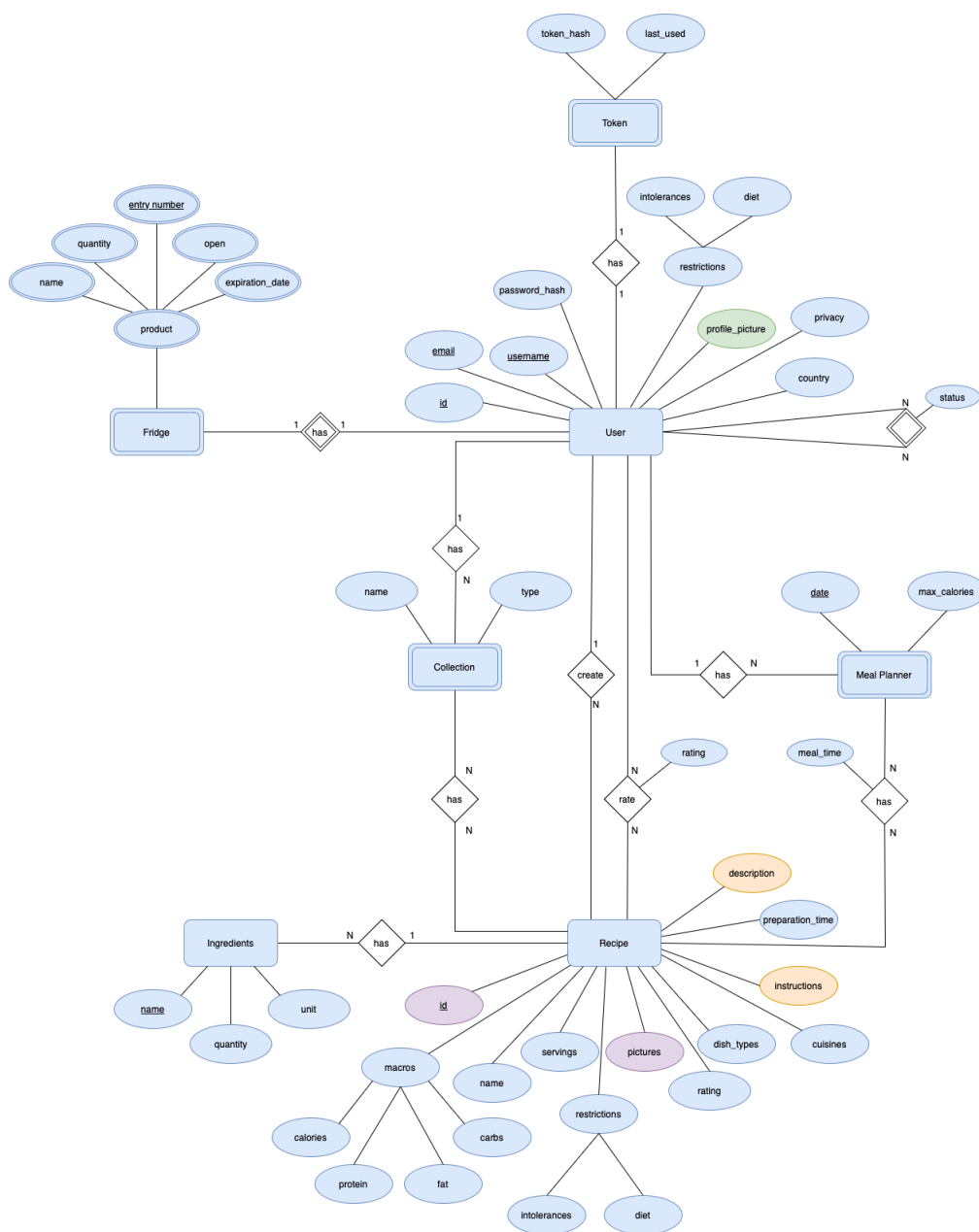


Figura A.1: Modelo Entidade-Associação

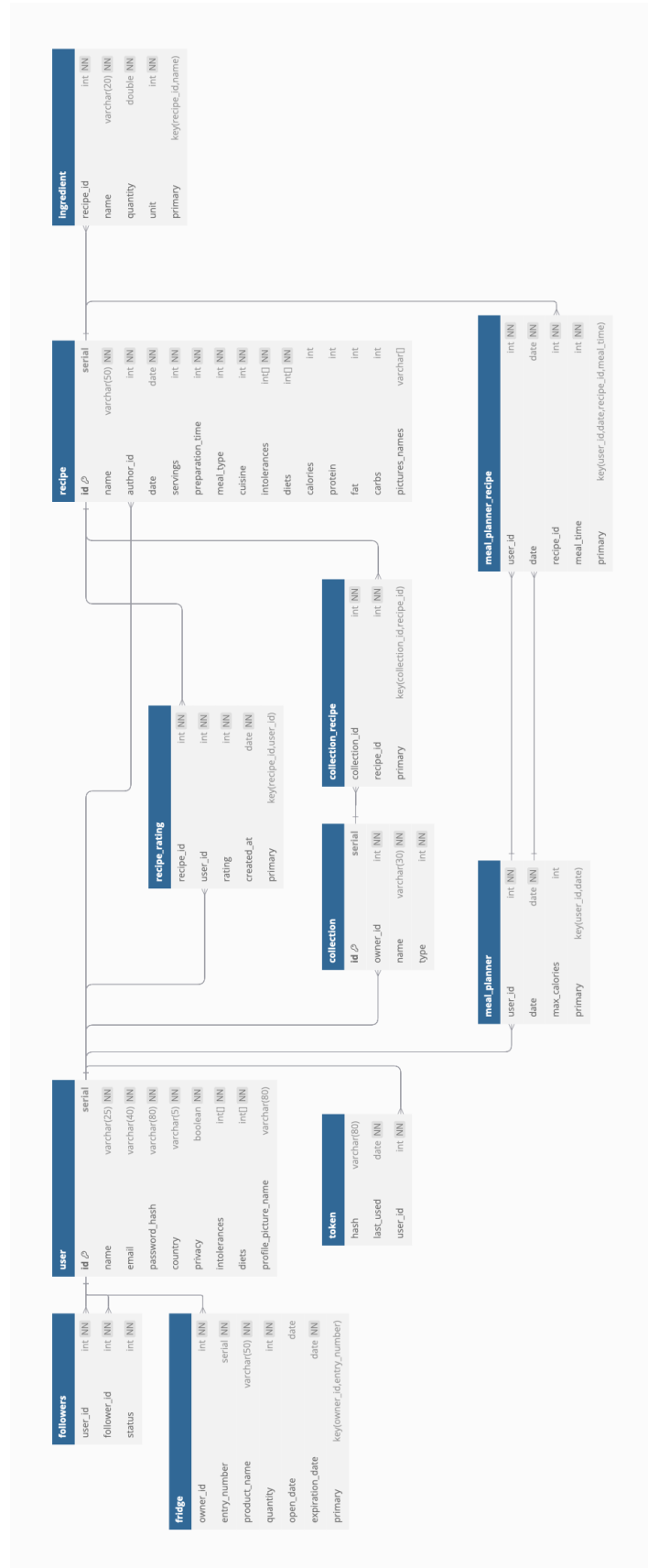


Figura A.2: Modelo Físico